

BÀI 07: VALIDATION – PIPE TRONG NESTJS

Mục tiêu bài 07: Bài này sẽ giới thiệu đến các bạn về Validation - Pipe trong NestJS và cách sử dụng Validation – Pipe trong dự án NestJS theo từng bước như thế nào thông qua ví dụ cụ thể.

7.1. GIỚI THIỆU VALIDATION

Trong ứng dụng **web/API**, **validation** giúp đảm bảo dữ liệu đầu vào từ **client** (người dùng) là đúng định dạng, hợp lệ, từ đó:

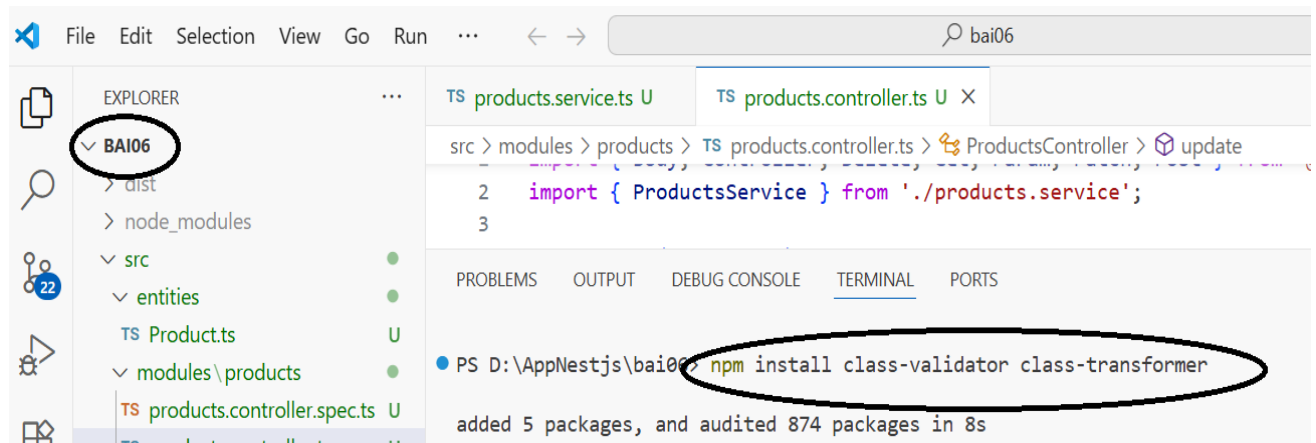
- Tránh lỗi hoặc hành vi không mong muốn
- Đảm bảo tính bảo mật và toàn vẹn dữ liệu
- Cải thiện trải nghiệm người dùng bằng thông báo lỗi rõ ràng

7.2. CÀI ĐẶT VALIDATION

NestJS hỗ trợ **validation** thông qua 2 thư viện chính:

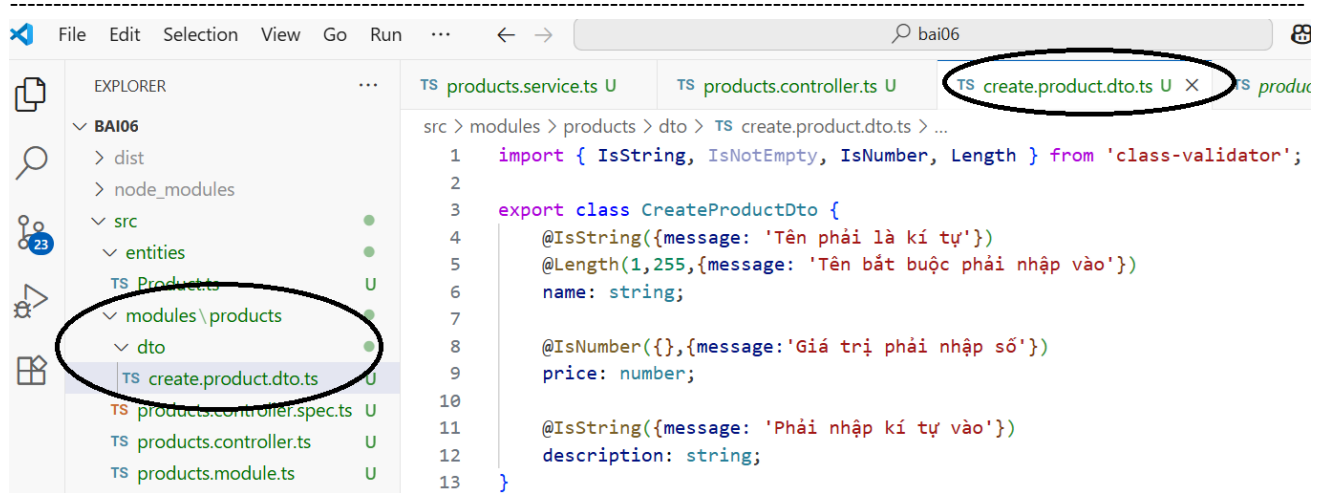
Thư viện	Vai trò
class-validator	Thêm các decorator để xác thực
class-transformer	Biến đổi dữ liệu từ plain JS object → class instance

Bây giờ chúng ta cài đặt **validation** vào dự án của **Bài 06**, các bạn mở **Terminal** gõ dòng lệnh: **npm install class-validator class-transformer**



7.3. TẠO DTO (Data Transfer Object) ĐỂ XÁC THỰC DỮ LIỆU

- Các bạn tạo một thư mục **dto** trong thư mục **products** của dự án, tạo tập tin **create.product.dto.ts** trong thư mục **dto** và viết code như sau:



- Chúng ta đã ràng buộc dữ liệu khi nhập vào các trường: **name, price, description**.

- Một số **decorator** thường dùng:

Decorator	Ý nghĩa
@IsString()	Kiểu chuỗi
@IsNumber()	Kiểu số
@IsInt()	Số nguyên
@IsBoolean()	Kiểu boolean
@IsNotEmpty()	Không được rỗng
@IsOptional()	Không bắt buộc
@IsEmail()	Phải đúng định dạng email
@Length(x, y)	Độ dài chuỗi từ x đến y
@Min(x), @Max(x)	Giá trị nhỏ nhất/lớn nhất
@Matches(regex)	Phù hợp với biểu thức chính quy

7.4. SỬ DỤNG DTO TRONG CONTROLLER

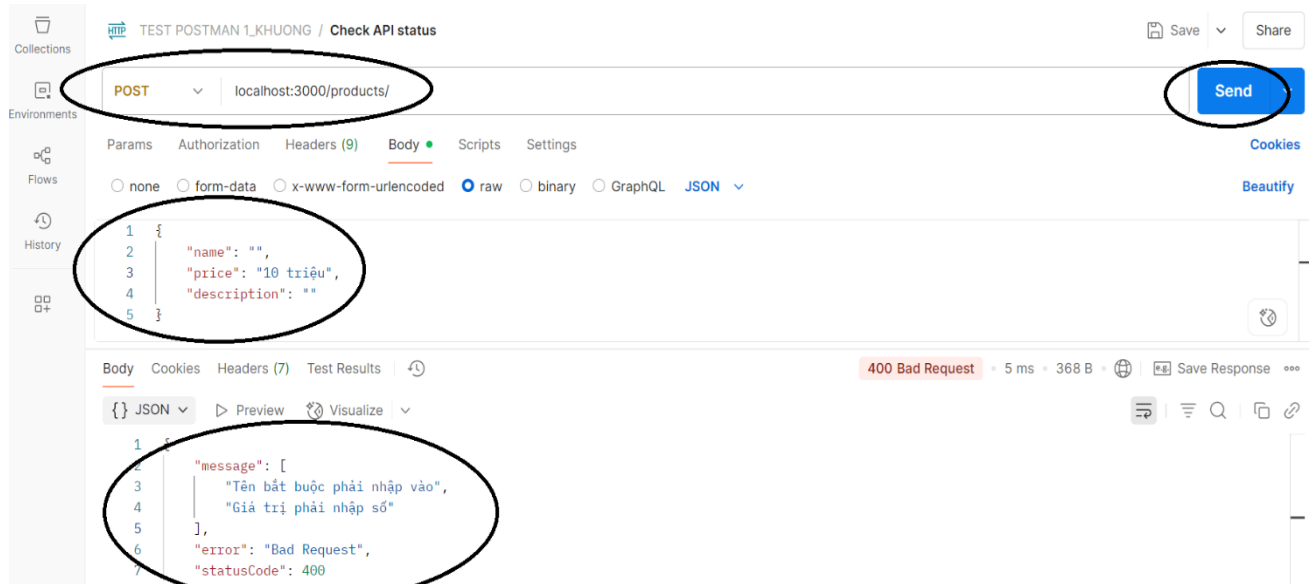
- Để sử dụng **create.product.dto.ts** đã tạo các bạn mở lại tập tin **products.controller.ts** sửa lại code cho hàm: **create ()** như bên dưới:



7.5. BIÊN DỊCH VÀ KIỂM TRA VALIDATION

- Các bạn mở Terminal, gõ lệnh biên dịch chương trình: **nest start npm run start:dev** (Nhớ là phải chạy Xampp để phục vụ cơ sở dữ liệu nhé)

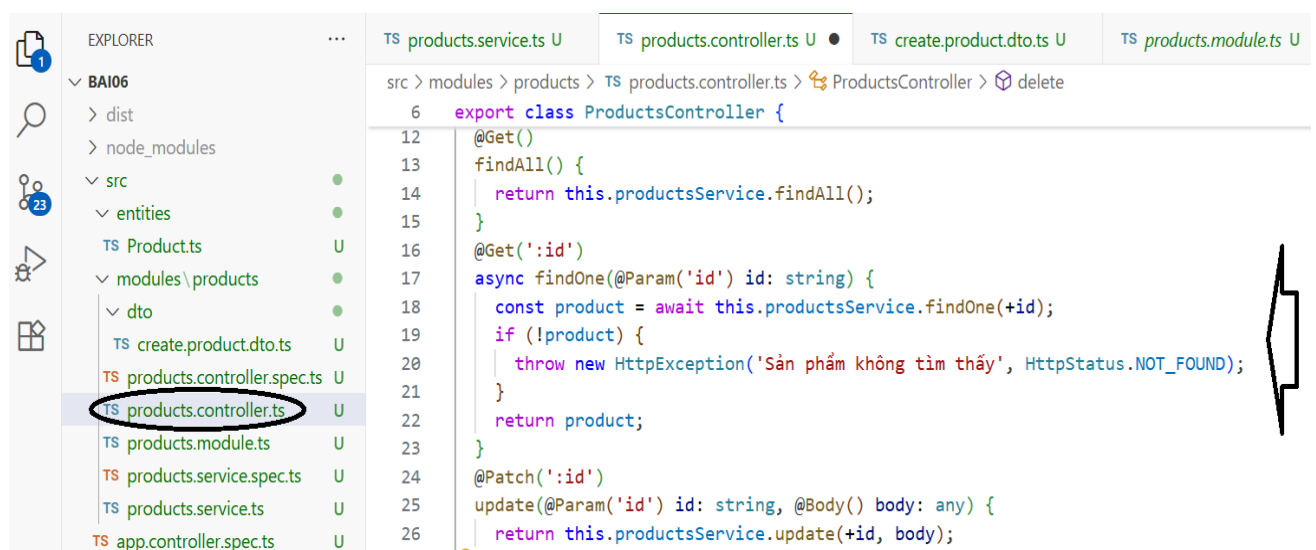
- Mở chương trình **Postman**, chọn phương thức **POST**, gõ đường dẫn: **localhost:3000/products/**, gõ dữ liệu nhập vào bị lỗi như bên dưới. Khi chọn **Send** sẽ thông báo lỗi nhập dữ liệu theo **Validation** đã tạo.



7.6. TẠO VALIDATION TRONG HÀM CONTROLLER

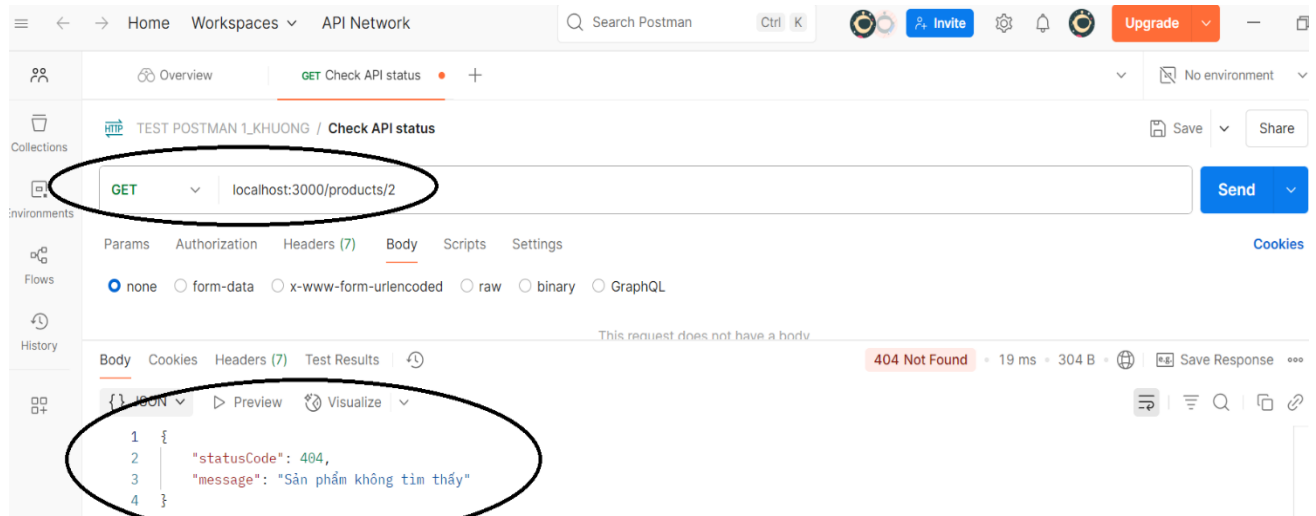
- Các bạn có thể tạo điều kiện bằng cách **viết hàm** ngay trong hàm **Controller**, ví dụ chúng ta muốn hiển thị Sản phẩm mà không có trong cơ sở dữ liệu thì sẽ thông báo lỗi.

- Các bạn mở tập tin **products.controller.ts** và sửa code của hàm **findOne()** như bên dưới:



- Biên dịch lại chương trình: **nest start npm run start:dev**

- Mở chương trình **Postman**, chọn phương thức **GET**, gõ đường dẫn: **localhost:3000/products/2**, khi chọn **Send** sẽ thấy thông báo lỗi vì trong cơ sở dữ liệu không có id sản phẩm nào =2



7.7. VẤN ĐỀ NÂNG CAO VỚI VALIDATE

7.7.1. Tạo validate Global

- Muốn tạo **Validate** cho toàn bộ ứng dụng, lúc đó sẽ tự động áp dụng **Validate** cho tất cả các **Controller** của ứng dụng thì các bạn sử dụng **Validate Global**.

- Để tạo **Validtate Global** các bạn mở tập tin **main.ts** và viết thêm đoạn code sau:

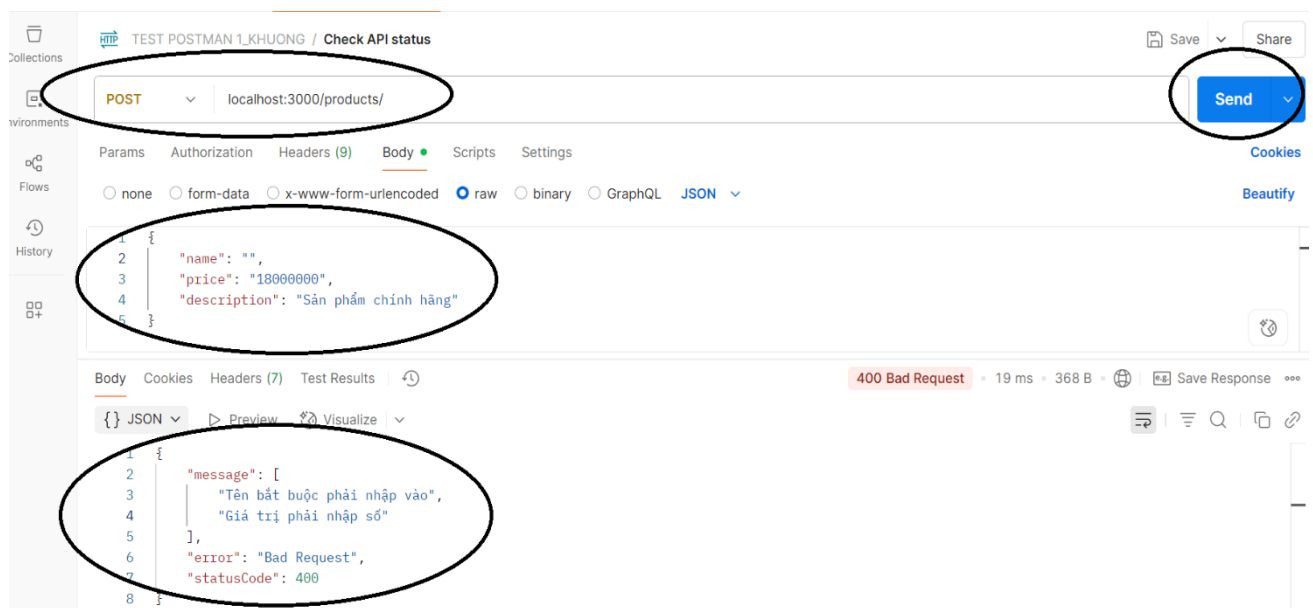


- Bây giờ các bạn mở lại tập tin **products.controller.ts** và sửa lại đoạn code hàm **create ()** (bỏ **ValidationPipe** đi) như bên dưới:



- Biên dịch lại chương trình: **nest start npm run start:dev**

- Mở chương trình **Postman** ra và thiết lập để chạy phương thức **POST** (**localhost:3000/products/**), gõ nội dung đưa sản phẩm vào có thể báo lỗi như bên dưới:



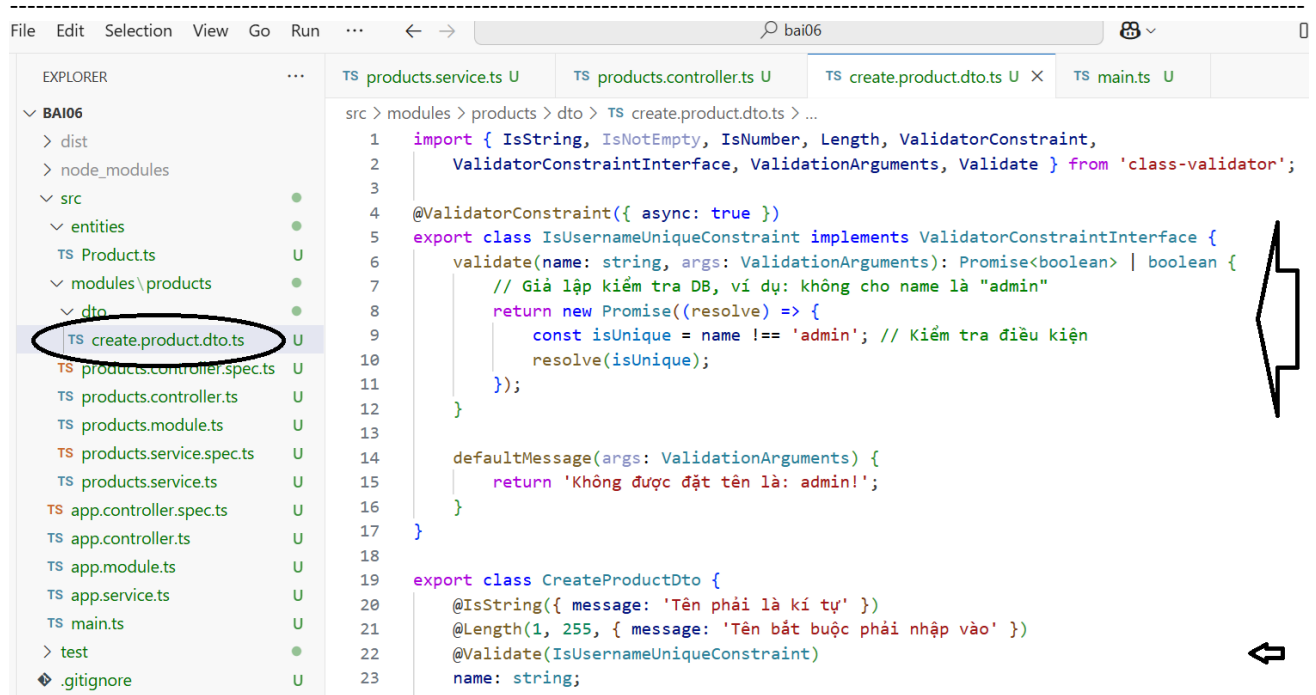
- Thì chúng ta thấy chương trình vẫn báo lỗi khi nhập dữ liệu vào. Có nghĩa là mọi **Service** trước khi gọi vào **Controller** thì đều phải thông qua điều **Validation** trong **dto**.

7.7.2. Custom (tự định nghĩa) Validation

- Chúng ta hoàn toàn có thể **Custom** (tự định nghĩa) các **Validation** (điều kiện) của các bạn theo yêu cầu của mình.

- Ví dụ bây giờ chúng ta sẽ **Custom** điều kiện nhập dữ liệu cho trường **name** là không được nhập chữ: **“admin”** bằng cách: Các bạn mở tập tin **create.product.dto.ts** viết thêm **class IsUsernameUniqueConstraint** và gọi vào điều kiện nhập dữ liệu trường **name** như bên dưới:

Bài 07: Validation - Pipe trong NestJS



- Biên dịch lại chương trình: **nest start npm run start:dev**

- Mở chương trình **Postman** ra và thiết lập để chạy phương thức **POST** (**localhost:3000/products/**), gõ nội dung đưa sản phẩm vào có thể báo lỗi như bên dưới:

